

PHASE 4B – Training Data Generation & Auto-Labeling (FULL TECHNICAL DOCUMENT)

1. Phase Objective

Phase 4B converts the **deterministic rule-based AI engine (Phase 4A)** into a **training-data generator**. This phase does NOT train any ML model. Instead, it produces **clean, explainable, reproducible labeled data** using weak supervision.

2. Why Phase 4B Is Required

ML models require labeled data. Manual labeling of resumes is impractical.

Hence we use:

```
Rule-Based Engine (Phase 4A)
  ↓
Batch Resume Processing
  ↓
Auto-Generated Scores & Labels
  ↓
Training Dataset
```

This approach is called **weak supervision** and is industry standard.

3. Dataset Details

- Source: Kaggle Resume Dataset
- Size: ~2500 resumes
- Formats: PDF, DOCX
- Local path used:

```
E:\Projects\UTAP\temp
```

All resumes were merged into a **single folder** to simulate real ATS intake systems.

4. Architectural Isolation

To avoid breaking previous phases:

```
ai_engine/
├── services/           ← Phase 4A (stable logic)
├── experiments/       ← Phase 4B (experimental)
└── management/
```

Only `experiments/` is modified in Phase 4B.

5. Folder Structure Created

```
ai_engine/
├── experiments/
│   ├── resume_parser.py
│   ├── dataset_loader.py
│   ├── job_definitions.py
│   ├── batch_runner.py
│   └── metrics.py
├── management/
│   └── commands/
│       └── run_phase4b.py
```

6. Libraries Installed (Experiments Only)

```
pip install PyPDF2 python-docx
```

Used strictly for text extraction, never for intelligence.

7. Resume Parsing Code

File: `ai_engine/experiments/resume_parser.py`

```
from PyPDF2 import PdfReader
from docx import Document

def parse_pdf(file_path):
    text = ''
    reader = PdfReader(file_path)
    for page in reader.pages:
        if page.extract_text():
            text += page.extract_text() + ' '
    return text.strip()
```

```

def parse_docx(file_path):
    text=''
    doc=Document(file_path)
    for para in doc.paragraphs:
        text+=para.text+' '
    return text.strip()

def parse_resume(file_path):
    if file_path.lower().endswith('.pdf'):
        return parse_pdf(file_path)
    if file_path.lower().endswith('.docx'):
        return parse_docx(file_path)
    return ''

```

8. Dataset Loader Code

File: ai_engine/experiments/dataset_loader.py

```

import os
from ai_engine.experiments.resume_parser import parse_resume
from ai_engine.services.text_utils import normalize_text
from ai_engine.services.skill_extractor import extract_skill_candidates
from ai_engine.services.registry import update_from_resume
from ai_engine.models import Resume

def load_resumes(folder_path,limit=None):
    processed=0
    for filename in os.listdir(folder_path):
        if limit and processed>=limit:
            break
        file_path=os.path.join(folder_path,filename)
        if not os.path.isfile(file_path):
            continue
        if not (filename.endswith('.pdf') or filename.endswith('.docx')):
            continue
        raw_text=parse_resume(file_path)
        if not raw_text:
            continue
        normalized=normalize_text(raw_text)
        skills=extract_skill_candidates(normalized)
        update_from_resume(skills)
        Resume.objects.create(
            candidate_id=1,
            file=file_path,

```

```
        raw_text=raw_text
    )
    processed+=1
    return processed
```

9. Hardcoded Job Definitions

File: `ai_engine/experiments/job_definitions.py`

```
HARDCODED_JOBS={
    'backend_developer':{
        'skills':['python','django','sql','apis'],
        'min_score':30
    },
    'data_scientist':{
        'skills':['python','ml','statistics','pandas'],
        'min_score':35
    },
    'frontend_developer':{
        'skills':['javascript','react','css'],
        'min_score':30
    },
    'devops_engineer':{
        'skills':['docker','kubernetes','ci','linux'],
        'min_score':35
    }
}
```

10. Batch Scoring Code

File: `ai_engine/experiments/batch_runner.py`

```
from ai_engine.models import Resume,ResumeJobScore
from ai_engine.experiments.job_definitions import HARDCODED_JOBS
from ai_engine.services.scoring import calculate_match_score
from ai_engine.services.text_utils import normalize_text
from ai_engine.services.skill_extractor import extract_skill_candidates

def run_batch_scoring():
    resumes=Resume.objects.all()
    for resume in resumes:
        normalized=normalize_text(resume.raw_text)
        resume_skills=extract_skill_candidates(normalized)
        for job_name,job in HARDCODED_JOBS.items():
```

```
score=calculate_match_score(job['skills'],resume_skills)
eligible=score>=job['min_score']
ResumeJobScore.objects.create(
    resume=resume,
    job_name=job_name,
    score=score,
    eligible=eligible
)
```

11. Controller Command

File: ai_engine/management/commands/run_phase4b.py

```
from django.core.management.base import BaseCommand
from ai_engine.experiments.dataset_loader import load_resumes
from ai_engine.experiments.batch_runner import run_batch_scoring

class Command(BaseCommand):
    help='Run Phase 4B batch evaluation pipeline'

    def handle(self,*args,**kwargs):
        count=load_resumes(r'E:\\Projects\\UTAP\\temp',limit=20)
        run_batch_scoring()
        self.stdout.write(f'Phase 4B auto-labeling completed on {count}
resumes')
```

12. Execution Command

```
python manage.py run_phase4b
```

13. Validation Results

Verified using Django shell:

```
Resume.objects.count() # 20
ResumeJobScore.objects.count() # 80
```

14. Phase 4B Outcome

Phase 4B successfully generated a **training-ready, explainable, reproducible labeled dataset** using weak supervision, bridging deterministic logic and future ML/NLP training phases.

PHASE 4B STATUS: COMPLETE